Combinatorial Algorithms
HW2: Image unshredding

Industrial Informatics Department
Czech Technical University in Prague
https://industrialinformatics.fel.cvut.cz/

March 18, 2026

1 Lazy Constraints

Since combinatorial optimization problems are studied for decades already, it is natural that there exist a lot of advanced optimization approaches that result in reduced computation time for certain problem types. One of the conceptually easiest approach is called *lazy constraints* [1]. The basic idea is to initially formulate the problem only with the most essential constraints, omitting those that are only rarely violated. These other constraints are checked and added one-by-one to the model only if the current solution violates any of them. In other words, some constraints are generated in a lazy fashion, i.e. the constraint is added to the model only if the solution violates it.

The application of lazy constraints to TSP is really straightforward. Let model TSP as follows

$$\min \sum_{(i,j)\in E} c_{i,j} \cdot x_{i,j} \tag{1}$$

$$\text{s.t.} \sum_{(i,j)\in E} x_{i,j} = 1, \quad j \in V \tag{2}$$

$$\sum_{(i,j)\in E} x_{i,j} = 1, \quad i \in V \tag{3}$$

$$\sum_{i,j\in S: i\neq j} x_{i,j} \leq |S| - 1, \quad S \subset V, S \neq \emptyset \tag{4}$$

$$x_{i,j} \in \{0,1\} \tag{5}$$

Figure 1: ILP formulation of TSP, suitable for lazy constraints.

It requires that in each proper non-empty subset of vertices S there are at most $|S| - 1$ edges between them. Indeed, having a subset of vertices with $|S|$ edges if $S \neq V$ means that there are subtours in the solution, for instance if $S = \{1, 2, 3\}$ then $\sum_{i,j\in S: i\neq j} x_{i,j} = 3 > |S| - 1$.

The problem with model in Fig.1 is that there are exponential number of subsets S, e.g. $n = 10$ results in $2^{n-2} = 1022$ constraints of (4).

However, we may generate Constraint (4) in a lazy manner. In the beginning, the problem is formulated first without these constraints, i.e. only with Constraints (2) and (3). Then each time when the new integer solution is found during branching, we find a cycle in this solution; let S be the nodes in this cycle. If $|S| < n$, i.e. the solution contains a subtour, the corresponding constraint (4) is added to the model. Adding this constraint excludes this solution from the search

space in the further run. The constraints are added to the model until the solution is the cycle of length n.

1.1 Lazy Constraints in Gurobi

Modern solvers, including Gurobi optimizer, allow the user to apply the lazy constraints trick by using so called *callbacks* that are called whenever a feasible solution is found.

1.1.0.1 Python Lazy constraints have to be enabled by setting model parameter `lazyConstraints` to 1.

```
model = g.Model()
model.Params.lazyConstraints = 1
```

To define a callback, create a function that accepts two arguments: `model` and `where`. The callback is then passed to model when calling `optimize()`.

```
def my_callback(model, where):
    # Callback is called when some event occur. The type of event is
    # distinguished using argument ''where''.
    # In this case, we want to perform something when an integer
    # solution is found, which corresponds to ''GRB.Callback.MIPSOL''.
    if where == GRB.Callback.MIPSOL:
        # TODO: your code here...

        # Get the value of variable x[i, j] from the solution.
        # You may also pass a list of variables to the method.
        value = model.cbGetSolution(x[i, j])

        # Add lazy constraint to model.
        model.cbLazy(...)

model.optimize(my_callback)
```

1.1.0.2 C++ Lazy constraints have to be enabled by setting model parameter `GRB.IntParam.LazyConstraints` to 1.

```
GRBModel model(env);
model.set(GRB_IntParam_LazyConstraints, 1);
```

To define a callback, inherit from class `GRBCallback` and override method `callback()`. The callback is then passed to model by `setCallback()`.

```
class MyCallback : public GRBCallback {
  protected:
    void callback() {
      // Callback is called when some event occur. The type of event is
      // distinguished using variable ''where'' defined in parent class.
      // In this case, we want to perform something when an integer
      // solution is found, which corresponds to ''GRB_CB_MIPSOL''.
      if (where == GRB_CB_MIPSOL) {
        // TODO: your code here...

        // Get the value of variable x[i, j] from the solution.
        // There are also methods for getting multiple values at once, see docs.
        double value = getSolution(x[i, j]);

        // Add lazy constraint to model.
        addLazy(...);
      }
    }
}

MyCallback cb;
model.setCallback(&cb);
model.optimize();
```

2 A homework assignment - image unshredding

Image unshredding is a problem of reconstructing shredded images from a set of stripes as closely as possible to the original image. For example, Fig. 2 shows the shredded image and Fig. 3 is its reconstruction.

Figure 2: Example of shredded image.

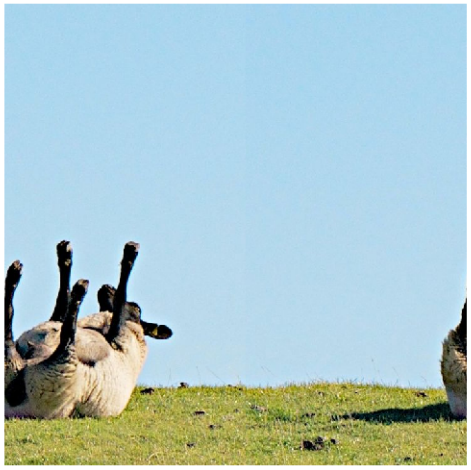
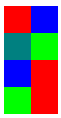


Figure 3: Example of reconstructed image.

Interestingly, quite reliable image reconstructions can be obtained by transforming this problem to Shortest Hamiltonian Path Problem (SHPP), where each stripe will represent one node in a graph. The idea is to order the stripes in such a way that adjacent stripes are "similar". For this, we need to define a distance function between two stripes.

Let w denote the width, i.e., the number of pixel columns in each stripe and let h denote the height, i.e., the number of rows in each stripe. Assume that we have two stripes $S^{(1)}$ (see Fig. 4) and $S^{(2)}$ (see Fig. 5). Both stripes have width $w = 2$ and height $h = 4$.

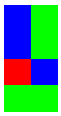


(a) Color image of the stripe.

$$\begin{bmatrix} 255 & 0 & 0 \\ 0 & 128 & 128 \\ 0 & 0 & 255 \\ 0 & 255 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 255 \\ 0 & 255 & 0 \\ 255 & 0 & 0 \\ 255 & 0 & 0 \end{bmatrix}$$

(b) RGB values of the pixels in left and right column.

Figure 4: Stripe $S^{(1)}$.



(a) Color image of the stripe.

$$\begin{bmatrix} 0 & 0 & 255 \\ 0 & 0 & 255 \\ 255 & 0 & 0 \\ 0 & 255 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 255 & 0 \\ 0 & 255 & 0 \\ 0 & 0 & 255 \\ 0 & 255 & 0 \end{bmatrix}$$

(b) RGB values of the pixels in left and right column.

Figure 5: Stripe $S^{(2)}$.

The distance function is a sum of the absolute difference between the RGB color components of the adjacent pixel columns of the stripes, i.e.,

$$\mathrm{dist}(S^{(1)}, S^{(2)}) = \sum_{i=1}^{h} \sum_{c=1}^{3} |s^{(1)}_{i,w,c} - s^{(2)}_{i,1,c}| \tag{6}$$

Notice that the distance function is generally not symmetric since different columns are compared, i.e.

$$\mathrm{dist}(S^{(1)}, S^{(2)}) = 1020 \tag{7}$$

$$\mathrm{dist}(S^{(2)}, S^{(1)}) = 765 \tag{8}$$

Therefore, the image unshredding problem can be solved by computing the distances between every pair of stripes and solving the corresponding SHPP. The solution of the SHPP then represents the order of the stripes.

Moreover, we can further transform SHPP to TSP by adding a "dummy" node that has a zero distance to all other nodes (the nodes that will be connected to the dummy node in the TSP solution are the left and right edges of the reconstructed image.). The motivation for this transformation is that there exists really good solvers for TSP, whereas for SHPP this is not the case.

A homework assignment: Implement a program that solves the image unshredding problem using TSP. Implement exact TSP solver using **lazy constraints**. Upload your source code to the BRUTE system where it will be automatically evaluated.

Hint 1 (for Python): Computing the distances between stripes in Python may lead to time-outs if not implemented efficiently. We strongly recommend using `numpy`, which is an optimized library for working with matrices using Matlab-like interface. Eq. (6) can then be computed using vectorized notation instead of two nested for-loops.

Hint 2: Use the following interface to your TSP solver: `int[] solveTsp(double[][] distances)`, where `distances[i][j]` is a distance from node i to node j. The result of the function is an ordered list of node indices forming the optimal circuit.

Hint 3: When an integer solution with multiple subtours is found in the callback, add the constraint that forbids the shortest one. Experimentally, forbidding shortest subtours shows good results.

2.1 Input and Output Format

Your program will be called with two arguments: the first one is absolute path to input file and the second one is the absolute path to output file (the output file has to be created by your program).

The input file has the following form

$$
\begin{array}{lllllllll}
n & w & h & & & & & & \\
s^{(1)}_{1,1,1} & s^{(1)}_{1,1,2} & s^{(1)}_{1,1,3} & s^{(1)}_{1,2,1} & \dots & s^{(1)}_{1,w,3} & s^{(1)}_{2,1,1} & \dots & s^{(1)}_{h,w,3} \\
s^{(2)}_{1,1,1} & s^{(2)}_{1,1,2} & s^{(2)}_{1,1,3} & s^{(2)}_{1,2,1} & \dots & s^{(2)}_{1,w,3} & s^{(2)}_{2,1,1} & \dots & s^{(2)}_{h,w,3} \\
\vdots & & & & & & & & \\
s^{(n)}_{1,1,1} & s^{(n)}_{1,1,2} & s^{(n)}_{1,1,3} & s^{(n)}_{1,2,1} & \dots & s^{(n)}_{1,w,3} & s^{(n)}_{2,1,1} & \dots & s^{(n)}_{h,w,3}
\end{array}
$$

One space is used as a separator between values on one line. All the values in the input file are non-negative integers. Values $s^{(k)}_{i,j,c} \in \{0, 1, \dots, 255\}$ represent the color components of the pixels in stripes.

The output file has the following form

$$\pi(1) \quad \pi(2) \quad \dots \quad \pi(n)$$

where π is an optimal permutation of the stripes (without the dummy stripe). All the values in the input file are positive integers.

Example 1

Notice that the first stripes correspond to stripes $S^{(1)}$ and $S^{(2)}$ from Fig. 4 and Fig. 5, respectively.

Input:

```
3 2 4
255 0 0 0 0 255 0 128 128 0 255 0 0 0 255 255 0 0 0 255 0 255 0 0
0 0 255 0 255 0 0 0 255 0 255 0 255 0 0 0 0 255 0 255 0 0 255 0
0 0 255 0 0 255 0 0 255 0 0 255 0 0 255 0 0 255 0 0 255 0 0 255
```

Output:

```
3 2 1
```

References

[1] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, *The traveling salesman problem: a computational study.* The traveling salesman problem: a computational study, 2011.